

# Laboratorio de prompts y métricas

PF1822 · Especialización en Desarrollo con IA

Módulo 2 · MA04576 · INTRODUCCIÓN A MODELOS GENERATIVOS Y CONSUMO POR API · 21 h

## TÉCNICA

Análisis de caso + simulación con tablero de puntajes

## TIEMPO ESTIMADO

6 h (1,5 h sincrónica + 4,5 h autónomas)

## INDICADORES DE LOGRO

3.1, 3.2, 3.3, 4.1, 4.2, 4.3

## APOYOS

Video interactivo "Del prompt a la respuesta"

## PRODUCTO

Notebook del pipeline + registro de prompts + tabla de métricas

## SE EVALÚA CON

Rúbrica de revisión de la solución (observación) (M2-Instrumento-1.pdf); Bitácora de registro de resultados y plan de trabajo (M2-Bitacora.pdf)

## APRENDIZAJES ESPERADOS

AE3. FORMULAR PROMPTS EFECTIVOS PARA MODELOS GENERATIVOS EN ESCENARIOS ZERO-SHOT O FEW-SHOT, CONSIDERANDO LA TAREA SOLICITADA Y LOS PARÁMETROS DE GENERACIÓN DISPONIBLES.

AE4. IMPLEMENTAR UN PIPELINE DE PREPROCESAMIENTO Y EVALUACIÓN BÁSICA DE TEXTO EN PYTHON, APLICANDO TÉCNICAS DE LIMPIEZA, NORMALIZACIÓN Y MÉTRICAS INICIALES DE CALIDAD, DE ACUERDO CON LOS REQUISITOS DE ENTRADA Y SALIDA DEL MODELO.

## 1 · Preprocesamiento (`preprocesar.py`):

```

"""Limpieza, normalización y tokenización de tickets antes de enviarlos al
modelo."""
import html
import re
import unicodedata

import spacy

nlp = spacy.load("es_core_news_sm") # python -m spacy download es_core_news_sm

def limpiar(texto: str) -> str:
    """Quita HTML, firma y datos personales; deja un texto listo para el modelo."""
    texto = html.unescape(texto)
    texto = re.sub(r"<br\s*/?>|</p>", "\n", texto, flags=re.IGNORECASE)
    texto = re.sub(r"<[^>]+>", " ", texto) # resto de etiquetas
    HTML
    texto = re.split(r"\n\s*--\s", texto)[0] # todo lo que sigue a
    la firma "-- "
    texto = re.sub(r"Enviado desde mi \w+", " ", texto, flags=re.IGNORECASE)
    texto = re.sub(r"\S+@\S+", "[correo]", texto) # datos personales
    fuera
    texto = re.sub(r"https?:\/\/\S+", "[enlace]", texto)
    texto = re.sub(r"!{2,}", "!", texto)
    return re.sub(r"\s+", " ", texto).strip()

def quitar_tildes(texto: str) -> str:
    """Pasa 'sesión' a 'sesion'; se usa solo para comparar textos al medir."""
    return "".join(c for c in unicodedata.normalize("NFD", texto) if
unicodedata.category(c) != "Mn")

def tokenizar(texto: str, lematizar: bool = False, sin_stopwords: bool = False) ->
list[str]:
    """Tokeniza con spaCy; opcionalmente lematiza y quita stopwords y puntuación."""
    tokens = []
    for t in nlp(texto.lower()):
        if t.is_punct or t.is_space or (sin_stopwords and t.is_stop):
            continue
        tokens.append(t.lemma_ if lematizar else t.text)
    return tokens

def ngramas(tokens: list[str], n: int = 2) -> list[tuple[str, ...]]:
    """Devuelve los n-gramas consecutivos de una lista de tokens."""
    return list(zip(*(tokens[i:] for i in range(n))))

```

*Ejemplo con T1:* `limpiar` deja "Hola equipo, Desde ayer en la tarde no puedo iniciar sesión en la app de facturación después de cambiar mi contraseña. Me dice "credenciales inválidas" aunque estoy segura de que la escribo bien. Saludos, Marta": se van las etiquetas y la firma con

el correo. `tokenizar(..., sin_stopwords=True)` deja, entre otros, `iniciar`, `sesión`, `app`, `facturación`, `contraseña`. El problema de T2 ("Compañía") **se conserva a propósito**: es parte del problema que reporta el cliente.

2 · Prompts de referencia:

Zero-shot:

```
[system] Eres analista de soporte de Nube Sur. Resume el ticket en UNA oración de máximo 25 palabras, en español neutro, en tercera persona. Indica qué falla y en qué producto. No agregues soluciones ni datos que no estén en el ticket.
[user] Ticket: <ticket limpio>
```

Few-shot: el mismo mensaje de sistema, seguido de dos ejemplos como turnos previos (`user` con un ticket y `assistant` con su resumen) antes del ticket a resumir. Ejemplos **distintos** de los cinco tickets evaluados, para no contaminar la medición:

```
[user] Ticket: No me llegan los correos de recuperación de clave desde el viernes.
[assistant] El usuario no recibe los correos de recuperación de clave desde el viernes.
[user] Ticket: ¿Cómo agrego un segundo administrador a mi cuenta?
[assistant] El usuario pregunta cómo agregar un segundo administrador a su cuenta.
[user] Ticket: <ticket limpio>
```

3 · Registro de iteraciones (ejemplo de nivel logrado):

| Versión    | Cambio                                                                                          | Efecto observado                                                                                                            |
|------------|-------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| v1         | "Resume esto: <ticket>"                                                                         | Resúmenes de 3 a 5 oraciones, a veces con soluciones inventadas. Falta <b>control</b> .                                     |
| v2         | Rol de analista + "una oración, máximo 25 palabras" + "no agregues soluciones"                  | Largo correcto; T4 sale como instrucción ("Exporta el listado...") en vez de pregunta. Mejora la <b>estructura</b> .        |
| v3         | "En tercera persona, indica qué falla y en qué producto" + few-shot con una pregunta de ejemplo | T4 queda "El usuario pregunta cómo...". Formato uniforme en los cinco. Mejora la <b>claridad</b> .                          |
| v3 · T=0,9 | Misma v3 con temperature 0,9                                                                    | Resúmenes más variados entre corridas y dos con palabras que no están en el ticket. Para esta tarea conviene <b>T=0,2</b> . |

4 · Medición (`evaluar.py`):

```

"""Mide resúmenes generados contra referencias y registra los resultados."""
import csv
import re

from nltk.translate.bleu_score import SmoothingFunction, sentence_bleu
from rouge_score import rouge_scorer

from preprocesar import quitar_tildes

class TokenizadorES:
    """rouge-score descarta por defecto las letras con tilde; este tokenizador las
    conserva."""

    def tokenize(self, texto: str) -> list[str]:
        return re.findall(r"\w+", quitar_tildes(texto.lower()))

tok = TokenizadorES()
rouge = rouge_scorer.RougeScorer(["rouge1", "rougeL"], tokenizer=tok)
suavizado = SmoothingFunction().method1 # textos cortos: sin suavizado BLEU da 0
con facilidad

def medir(referencia: str, generado: str) -> dict:
    """Devuelve ROUGE-1, ROUGE-L (F1) y BLEU de un resumen frente a su
    referencia."""
    r = rouge.score(referencia, generado)
    bleu = sentence_bleu([tok.tokenize(referencia)], tok.tokenize(generado),
    smoothing_function=suavizado)
    return {"rouge1_f": round(r["rouge1"].fmeasure, 3), "rougeL_f":
    round(r["rougeL"].fmeasure, 3),
            "bleu": round(bleu, 3)}

def registrar(filas: list[dict], ruta: str = "resultados.csv") -> None:
    """Escribe el protocolo: una fila por ticket, estrategia y temperature."""
    campos = ["ticket", "estrategia", "version", "temperature", "rouge1_f",
    "rougeL_f", "bleu",
            "tokens", "coherencia_1a3", "relevancia_1a3", "comentario"]
    with open(ruta, "w", newline="", encoding="utf-8") as f:
        escritor = csv.DictWriter(f, fieldnames=campos)
        escritor.writeheader()
        escritor.writerows(filas)

```

*Cálculo de comprobación (para verificar que la métrica está bien armada).* Referencia de T1: "El usuario no puede iniciar sesión en la app de facturación tras cambiar su contraseña." Resumen generado de ejemplo: "Usuario no logra iniciar sesión en la app de facturación después de cambiar la contraseña." Ambos tienen 15 palabras y comparten 11 (usuario, no, iniciar, sesión, en, la, app, de, facturación, cambiar, contraseña), que además aparecen en el mismo orden. Por lo tanto **ROUGE-1 F1 = ROUGE-L F1 = 11/15 = 0,733**. Si el participante obtiene un valor muy distinto con este par, su tokenizador está perdiendo las palabras con tilde.

**Protocolo de coherencia y relevancia** (valoración humana, 1 a 3): coherencia 3 = una oración gramatical y sin contradicciones; relevancia 3 = nombra la falla y el producto del ticket sin agregar nada que no esté en él. Las valora el participante y el tutor revisa una muestra.

#### 5 · Recomendación modelo:

Recomiendo few-shot v3 con temperature 0,2: obtuvo el ROUGE-L promedio más alto de las cuatro configuraciones, todos sus resúmenes tuvieron coherencia y relevancia 3, y fue la más estable entre corridas. El zero-shot quedó cerca en ROUGE pero falló el formato en T4. Con temperature 0,9 bajó la relevancia porque apareció información que no estaba en el ticket. Costo: el few-shot usa más tokens de entrada por los ejemplos; con 300 tickets diarios conviene medir si esa diferencia importa.

*(Los números de cada participante varían según el modelo y la corrida; lo que se evalúa es que la recomendación se apoye en su propia tabla.)*

**Qué distingue una respuesta débil:** elige la configuración "que se ve mejor" sin tabla; usa los mismos tickets de evaluación como ejemplos del few-shot; o reporta ROUGE calculado con el tokenizador por defecto, que en español descarta las letras con tilde.